

# Configuración de MLflow Tracking Server en AWS EC2

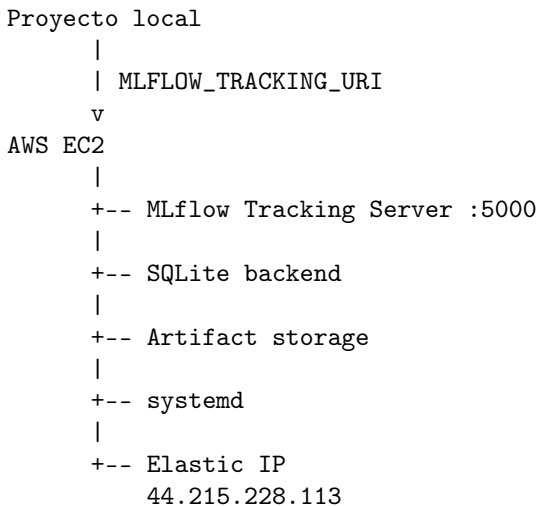
Issue #11 – MLOps

## 1. Objetivo

Configurar un servidor centralizado de MLflow Tracking sobre AWS EC2 y conectar el proyecto mediante la variable de entorno `MLFLOW_TRACKING_URI`. El servidor permite centralizar el registro de experimentos, métricas, parámetros y artefactos utilizados durante el desarrollo de los modelos de Machine Learning.

## 2. Arquitectura

La solución implementada utiliza el siguiente flujo:



## 3. Instancia AWS EC2

Se creó una instancia EC2 denominada *MLflow Tracking Server* utilizando Amazon Linux 2023 y una instancia `t3.small`, con 2 vCPU y 2 GiB de memoria.

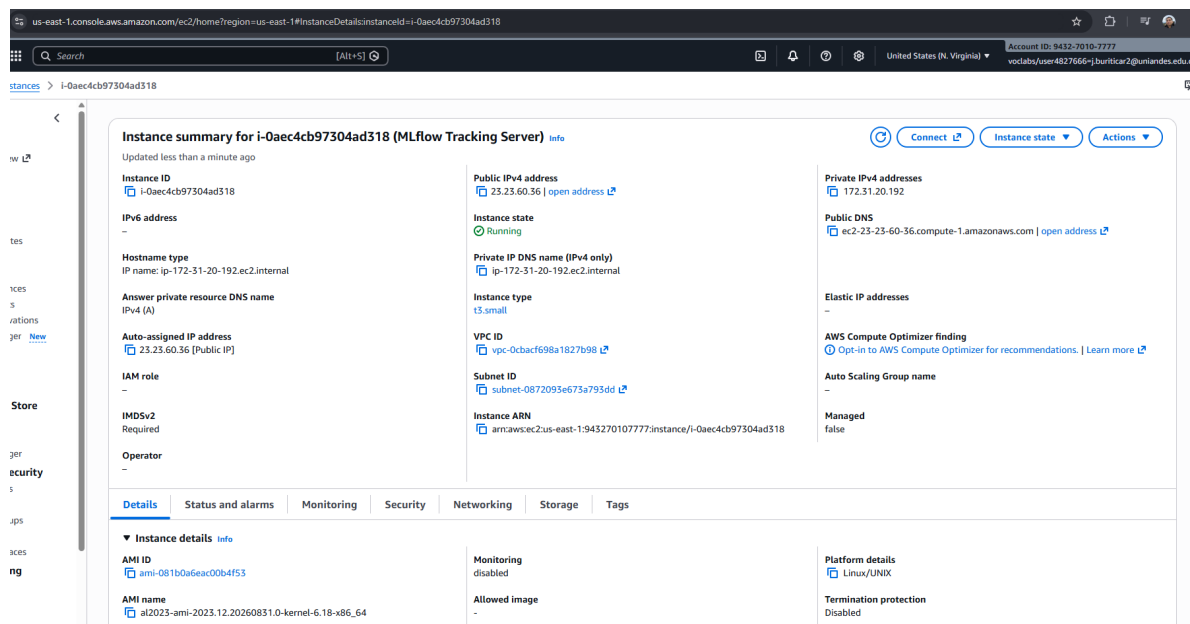


Figura 1: Instancia EC2 utilizada para el servidor MLflow.

## 4. Configuración de red

El Security Group de la instancia permite el acceso mediante:

- TCP 22 para administración mediante SSH.
- TCP 5000 para el acceso al MLflow Tracking Server.

El puerto 5000 es utilizado por MLflow para permitir las conexiones del proyecto y el acceso a su interfaz web.

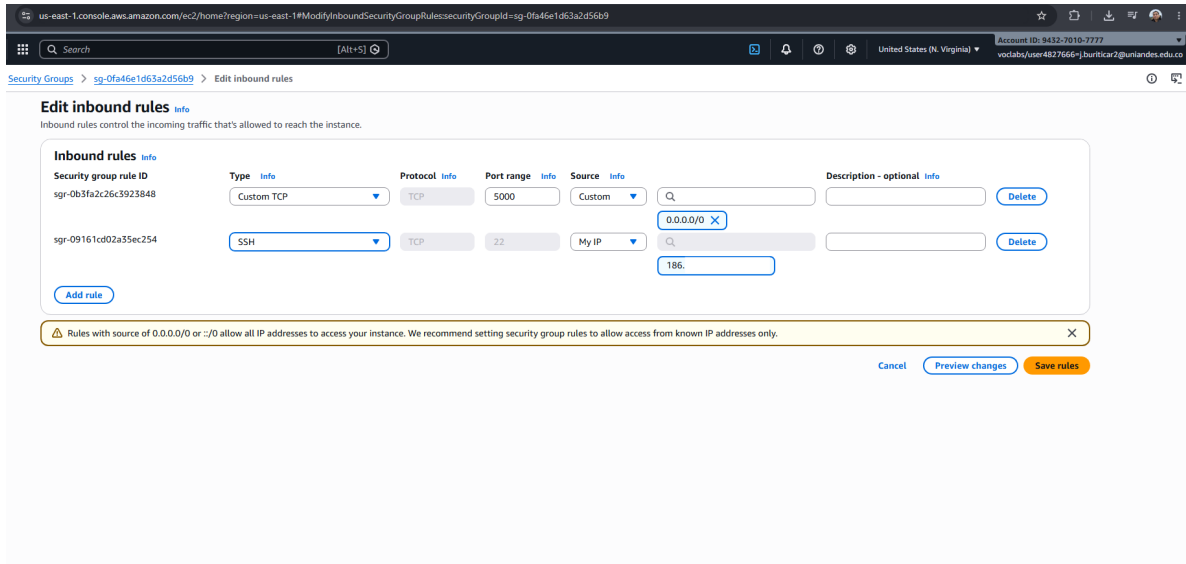


Figura 2: Reglas de entrada configuradas para la instancia EC2.

## 5. Acceso a la instancia

La administración del servidor se realizó mediante SSH utilizando el usuario `ec2-user` y una clave privada asociada a la instancia.

```
ec2-user@ip-172-31-20-192:~ X + v

C:\Users\jpbur\Downloads>cd %USERPROFILE%\Downloads

C:\Users\jpbur\Downloads>dir neurorisk-mlflow.pem
Volume in drive C is OS
Volume Serial Number is AE79-4788

Directory of C:\Users\jpbur\Downloads

09/01/2026 07:29 PM          1,678 neurorisk-mlflow.pem
                1 File(s)          1,678 bytes
                0 Dir(s) 665,826,783,232 bytes free

C:\Users\jpbur\Downloads>ssh -i "neurorisk-mlflow.pem" ec2-user@23.23.60.36
The authenticity of host '23.23.60.36 (23.23.60.36)' can't be established.
ED25519 key fingerprint is SHA256:V/9su7Y90s7RQzmAPfZ4+2zgOVSIx3CrxhiDg+nCWBU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '23.23.60.36' (ED25519) to the list of known hosts.

#_
~\  #####_      Amazon Linux 2023
~~~\#####\
~~~\###|
~~~\#/  _--  https://aws.amazon.com/linux/amazon-linux-2023
~~~V~'  '->
~~~~
~~~~
~~~~
~/m/'
[ec2-user@ip-172-31-20-192 ~]$
```

Figura 3: Conexión SSH exitosa con la instancia EC2.

## 6. Entorno Python

Se creó un entorno virtual independiente para MLflow utilizando Python 3.12:

```
python3.12 -m venv ~/mlflow-venv
source ~/mlflow-venv/bin/activate
```

La versión de Python utilizada fue Python 3.12.

```
ec2-user@ip-172-31-20-192:~ X + v

[ec2-user@ip-172-31-20-192 ~]$ python3.12 -m venv ~/mlflow-venv
[ec2-user@ip-172-31-20-192 ~]$ source ~/mlflow-venv/bin/activate
((mlflow-venv) ) [ec2-user@ip-172-31-20-192 ~]$ python --versionpython --version
unknown option --versionpython
usage: python [option] ... [-c cmd | -m mod | file | -] [arg] ...
Try 'python -h' for more information.
((mlflow-venv) ) [ec2-user@ip-172-31-20-192 ~]$ python --version
Python 3.12.14
((mlflow-venv) ) [ec2-user@ip-172-31-20-192 ~]$ |
```

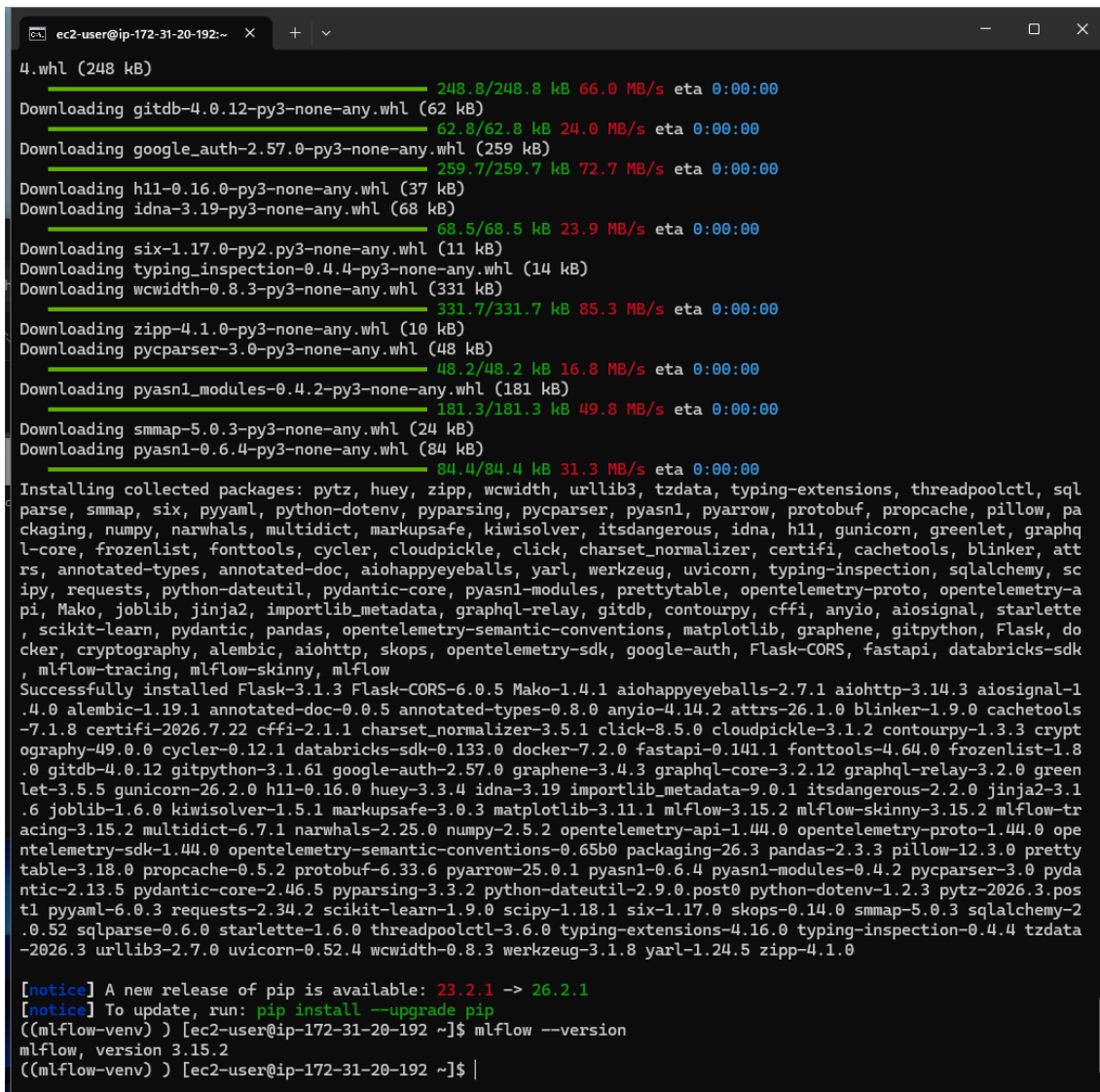
Figura 4: Entorno virtual de MLflow y Python 3.12.

## 7. Instalación de MLflow

MLflow fue instalado dentro del entorno virtual mediante:

```
pip install "mlflow>=2.13.0"
```

La versión instalada y utilizada fue MLflow 3.15.2.



```
4.whl (248 kB)
248.8/248.8 kB 66.0 MB/s eta 0:00:00
Downloading gitdb-4.0.12-py3-none-any.whl (62 kB)
62.8/62.8 kB 24.0 MB/s eta 0:00:00
Downloading google_auth-2.57.0-py3-none-any.whl (259 kB)
259.7/259.7 kB 72.7 MB/s eta 0:00:00
Downloading h11-0.16.0-py3-none-any.whl (37 kB)
Downloading idna-3.19-py3-none-any.whl (68 kB)
68.5/68.5 kB 23.9 MB/s eta 0:00:00
Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Downloading typing_inspection-0.4.4-py3-none-any.whl (14 kB)
Downloading wcwidth-0.8.3-py3-none-any.whl (331 kB)
331.7/331.7 kB 85.3 MB/s eta 0:00:00
Downloading zipp-4.1.0-py3-none-any.whl (10 kB)
Downloading pycparser-3.0-py3-none-any.whl (48 kB)
48.2/48.2 kB 16.8 MB/s eta 0:00:00
Downloading pyasn1_modules-0.4.2-py3-none-any.whl (181 kB)
181.3/181.3 kB 49.8 MB/s eta 0:00:00
Downloading smmap-5.0.3-py3-none-any.whl (24 kB)
Downloading pyasn1-0.6.4-py3-none-any.whl (84 kB)
84.4/84.4 kB 31.3 MB/s eta 0:00:00
Installing collected packages: pytz, huey, zipp, wcwidth, urllib3, tzdata, typing-extensions, threadpoolctl, sql
parse, smmap, six, pyyaml, python-dotenv, pyparsing, pycparser, pyasn1, pyarrow, protobuf, propcache, pillow, pa
chaging, numpy, narwhals, multidict, markupsafe, kiwisolver, itsdangerous, idna, h11, gunicorn, greenlet, graphq
l-core, frozenlist, fonttools, cycycler, cloudpickle, click, charset_normalizer, certifi, cachetools, blinker, att
rs, annotated-types, annotated-doc, aiohappyeyeballs, yarl, werkzeug, uvicorn, typing-inspection, sqlalchemy, sc
ipy, requests, python-dateutil, pydantic-core, pyasn1-modules, prettytable, opentelemetry-proto, opentelemetry-a
pi, Mako, joblib, jinja2, importlib_metadata, graphql-relay, gitdb, contourpy, cffi, anyio, aiosignal, starlette
, scikit-learn, pydantic, pandas, opentelemetry-semantic-conventions, matplotlib, graphene, gitpython, Flask, do
cker, cryptography, alembic, aiohttp, skops, opentelemetry-sdk, google-auth, Flask-CORS, fastapi, databricks-sdk
, mlflow-tracing, mlflow-skinny, mlflow
Successfully installed Flask-3.1.3 Flask-CORS-6.0.5 Mako-1.4.1 aiohappyeyeballs-2.7.1 aiohttp-3.14.3 aiosignal-1
.4.0 alembic-1.19.1 annotated-doc-0.0.5 annotated-types-0.8.0 anyio-4.14.2 attrs-26.1.0 blinker-1.9.0 cachetools
-7.1.8 certifi-2026.7.22 cffi-2.1.1 charset_normalizer-3.5.1 click-8.5.0 cloudpickle-3.1.2 contourpy-1.3.3 crypt
ography-49.0.0 cycycler-0.12.1 databricks-sdk-0.133.0 docker-7.2.0 fastapi-0.141.1 fonttools-4.64.0 frozenlist-1.8
.0 gitdb-4.0.12 gitpython-3.1.61 google-auth-2.57.0 graphene-3.4.3 graphql-core-3.2.12 graphql-relay-3.2.0 green
let-3.5.5 gunicorn-26.2.0 h11-0.16.0 huey-3.3.4 idna-3.19 importlib_metadata-9.0.1 itsdangerous-2.2.0 jinja2-3.1
.6 joblib-1.6.0 kiwisolver-1.5.1 markupsafe-3.0.3 matplotlib-3.11.1 mlflow-3.15.2 mlflow-skinny-3.15.2 mlflow-tr
acing-3.15.2 multidict-6.7.1 narwhals-2.25.0 numpy-2.5.2 opentelemetry-api-1.44.0 opentelemetry-proto-1.44.0 ope
ntelemetry-sdk-1.44.0 opentelemetry-semantic-conventions-0.65b0 packaging-26.3 pandas-2.3.3 pillow-12.3.0 pretty
table-3.18.0 propcache-0.5.2 protobuf-6.33.6 pyarrow-25.0.1 pyasn1-0.6.4 pyasn1-modules-0.4.2 pycparser-3.0 pyda
ntic-2.13.5 pydantic-core-2.46.5 pyparsing-3.3.2 python-dateutil-2.9.0.post0 python-dotenv-1.2.3 pytz-2026.3. pos
t0 pyyaml-6.0.3 requests-2.34.2 scikit-learn-1.9.0 scipy-1.18.1 six-1.17.0 skops-0.14.0 smmap-5.0.3 sqlalchemy-2
.0.52 sqlparse-0.6.0 starlette-1.6.0 threadpoolctl-3.6.0 typing-extensions-4.16.0 typing-inspection-0.4.4 tzdata
-2026.3 urllib3-2.7.0 uvicorn-0.52.4 wcwidth-0.8.3 werkzeug-3.1.8 yarl-1.24.5 zipp-4.1.0

[notice] A new release of pip is available: 23.2.1 -> 26.2.1
[notice] To update, run: pip install --upgrade pip
(mlflow-venv) [ec2-user@ip-172-31-20-192 ~]$ mlflow --version
mlflow, version 3.15.2
(mlflow-venv) [ec2-user@ip-172-31-20-192 ~]$ |
```

Figura 5: Instalación y verificación de MLflow 3.15.2.

## 8. Configuración del Tracking Server

El servidor MLflow fue configurado para escuchar en todas las interfaces de red sobre el puerto 5000. Se utilizó SQLite como almacenamiento de metadatos y un directorio local persistente para los artefactos.

La configuración utilizada fue:

```
mlflow server \
--host 0.0.0.0 \
--port 5000 \
--workers 1 \
--backend-store-uri sqlite:///home/ec2-user/mlflow/mlflow.db \
--default-artifact-root /home/ec2-user/mlflow/artifacts \
--allowed-hosts "44.215.228.113:5000"
```

El servidor utiliza un único worker debido a los recursos disponibles en la instancia t3.small.

## 9. MLflow Tracking Server en ejecución

El Tracking Server quedó operativo en EC2 y disponible mediante el puerto 5000.

```
ec2-user@ip-172-31-20-192:~$ mlflow server \
--host 0.0.0.0 \
--port 5000 \
--workers 1 \
--backend-store-uri sqlite:///home/ec2-user/mlflow/mlflow.db \
--default-artifact-root /home/ec2-user/mlflow/artifacts \
--allowed-hosts "23.23.60.36:5000"
Registry store URI not provided. Using backend store URI.
[MLflow] Security middleware enabled. Allowed hosts: 23.23.60.36:5000.
/home/ec2-user/mlflow-venv/lib64/python3.12/site-packages/mlflow/server/fastapi_app.py:19: StarletteDeprecationWarning: starlette.middleware.wsgi is deprecated and will be removed in a future release. Please refer to https://github.com/abersheeran/a2wsgi as a replacement.
  from starlette.middleware.wsgi import WSGIResponder, build_environ
2026/09/02 01:20:23 INFO: Started server process [26423]
2026/09/02 01:20:23 INFO: Waiting for application startup.
2026/09/02 01:20:23 INFO: Application startup complete.
2026/09/02 01:20:23 INFO: Uvicorn running on http://0.0.0.0:5000 (Press CTRL+C to quit)
2026/09/02 01:20:29 INFO: 186.155.174.164:53294 - "GET / HTTP/1.1" 200 OK
2026/09/02 01:20:29 INFO: 186.155.174.164:53294 - "GET /static-files/static/js/main.7a874c9f.js HTTP/1.1" 200 OK
2026/09/02 01:20:29 INFO: 186.155.174.164:58162 - "GET /static-files/static/css/main.b0f58ef9.css HTTP/1.1" 200 OK
2026/09/02 01:20:29 INFO: 186.155.174.164:55048 - "GET /static-files/manifest.json HTTP/1.1" 200 OK
2026/09/02 01:20:35 INFO: 186.155.174.164:53294 - "GET /static-files/TelemetryLogger.telemetry-worker.d0f8ea2c83c08b63528f.worker.js HTTP/1.1" 200 OK
2026/09/02 01:20:35 INFO: 186.155.174.164:53990 - "GET /static-files/favicon.ico HTTP/1.1" 200 OK
2026/09/02 01:20:35 INFO: 186.155.174.164:53664 - "GET /ajax-api/3.0/mlflow/ui-telemetry HTTP/1.1" 200 OK
```

Figura 6: MLflow Tracking Server ejecutándose en la instancia EC2.

## 10. Interfaz web de MLflow

La interfaz web de MLflow permite visualizar los experimentos y ejecuciones registradas en el Tracking Server.

El endpoint final configurado para el equipo es:

<http://44.215.228.113:5000>

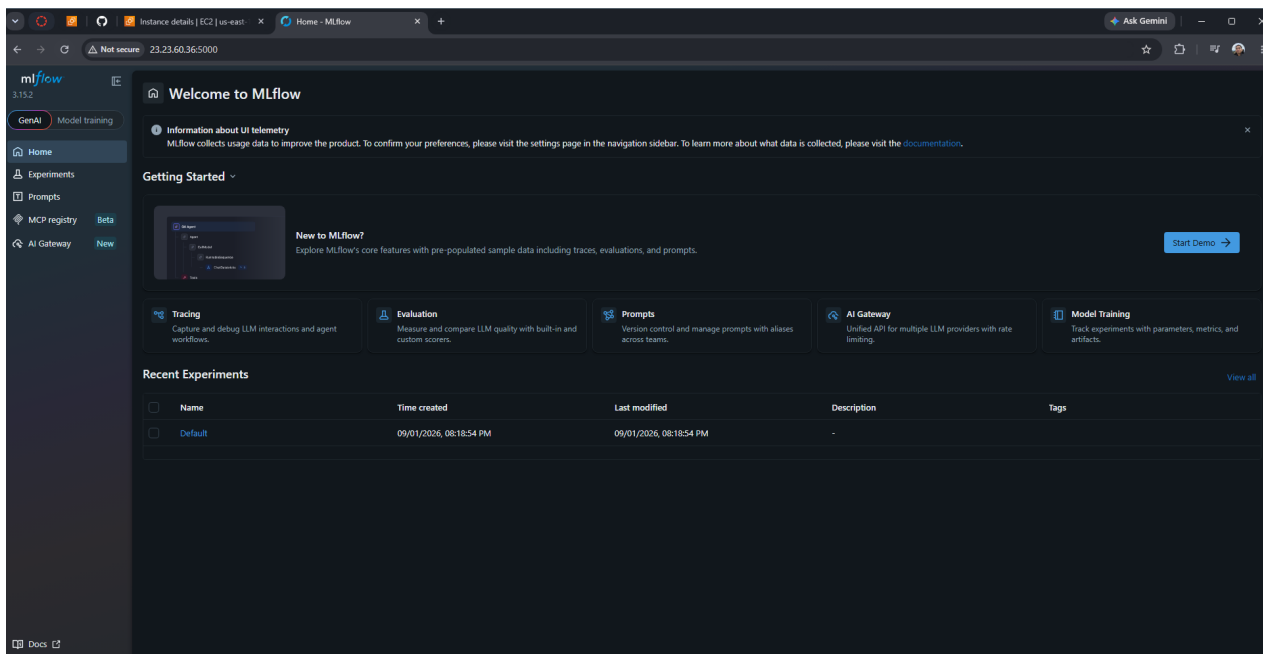
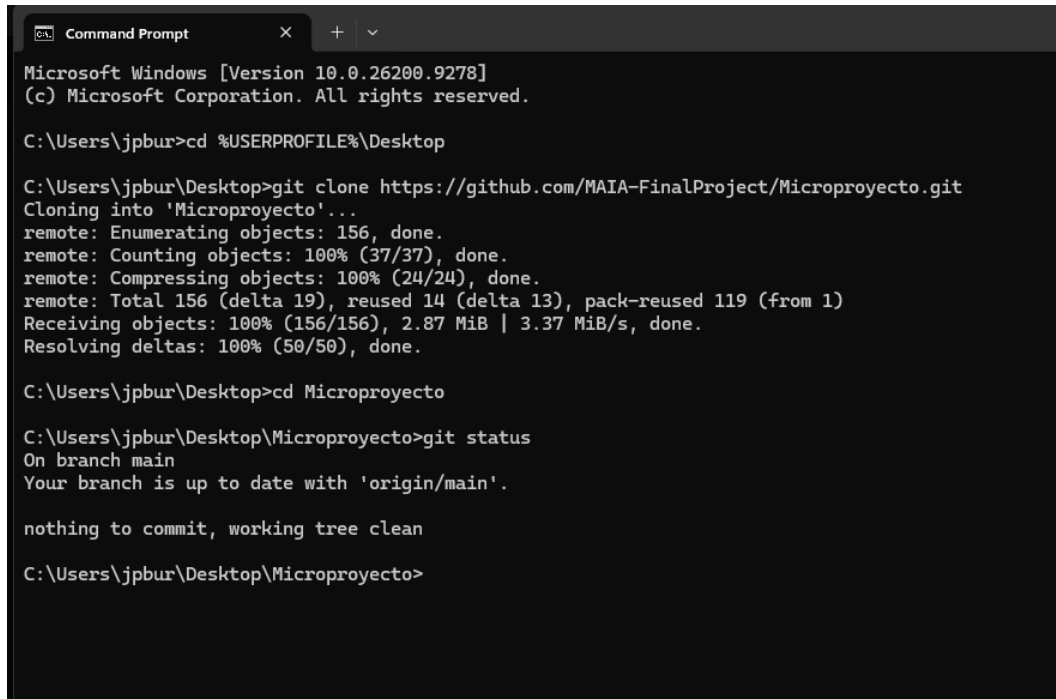


Figura 7: Interfaz web del MLflow Tracking Server.

## 11. Integración con el proyecto

El repositorio del proyecto fue clonado localmente para realizar la configuración y validación de la conexión con MLflow.



```
Microsoft Windows [Version 10.0.26200.9278]
(c) Microsoft Corporation. All rights reserved.

C:\Users\jpbur>cd %USERPROFILE%\Desktop

C:\Users\jpbur\Desktop>git clone https://github.com/MAIA-FinalProject/Microproyecto.git
Cloning into 'Microproyecto'...
remote: Enumerating objects: 156, done.
remote: Counting objects: 100% (37/37), done.
remote: Compressing objects: 100% (24/24), done.
remote: Total 156 (delta 19), reused 14 (delta 13), pack-reused 119 (from 1)
Receiving objects: 100% (156/156), 2.87 MiB | 3.37 MiB/s, done.
Resolving deltas: 100% (50/50), done.

C:\Users\jpbur\Desktop>cd Microproyecto

C:\Users\jpbur\Desktop\Microproyecto>git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

C:\Users\jpbur\Desktop\Microproyecto>
```

Figura 8: Repositorio del proyecto clonado localmente.

El proyecto utiliza `pydantic-settings` para cargar variables de entorno desde el archivo `.env`. En `src/config.py` se define:

```
mlflow_tracking_uri: str = "http://localhost:5000"

model_config = SettingsConfigDict(
    env_file=".env",
    env_file_encoding="utf-8",
    extra="ignore"
)
```

El valor utilizado para el servidor compartido se establece mediante:

```
MLFLOW_TRACKING_URI=http://44.215.228.113:5000
```

De esta manera, la dirección del Tracking Server no se encuentra hardcodeada en la lógica de la aplicación. El valor definido en `.env` sobrescribe el valor predeterminado de `localhost:5000`.

```
File Edit View

# Configuración del Entorno
ENVIRONMENT=development
LOG_LEVEL=INFO

# Google Drive Storage (DVC Remote)
# No requiere variables de entorno: la autenticación es OAuth individual por
# usuario. Al correr `dvc pull` o `dvc push` por primera vez se abre el
# navegador para iniciar sesión con la cuenta de Google invitada a la carpeta
# compartida del equipo; las credenciales quedan cacheadas localmente (fuera
# del repo), no en este archivo.

# MLflow Tracking Server

MLFLOW_TRACKING_URI=http://23.23.60.36:5000
MLFLOW_EXPERIMENT_NAME=preterm_infant_prediction

# Backend (FastAPI)
API_HOST=0.0.0.0
API_PORT=8000
API_URL=http://localhost:8000

# Frontend (Streamlit)
STREAMLIT_SERVER_PORT=8501
```

Figura 9: Configuración de MLFLOW\_TRACKING\_URI en el proyecto.

## 12. Elastic IP

Para mantener un endpoint estable después de detener y volver a iniciar la instancia EC2, se configuró una Elastic IP:

```
Elastic IP: 44.215.228.113
MLflow URL: http://44.215.228.113:5000
```

La Elastic IP se encuentra asociada a la instancia utilizada por el Tracking Server.

## 13. Inicio automático

Para evitar ejecutar manualmente MLflow después de cada reinicio de la instancia, se configuró un servicio systemd.

El servicio utiliza el entorno virtual de MLflow y ejecuta automáticamente el Tracking Server cuando la instancia EC2 inicia.

Los comandos utilizados para habilitar el servicio fueron:

```
sudo systemctl daemon-reload
sudo systemctl enable mlflow
sudo systemctl start mlflow
```

Con esta configuración, el flujo operativo es:

```
EC2 inicia
|
v
systemd inicia MLflow
|
v
```

```
MLflow :5000
|
v
44.215.228.113:5000
|
v
Proyecto mediante MLFLOW_TRACKING_URI
```

## 14. Validación

La configuración fue validada desde el proyecto mediante la clase **Settings**:

```
python -c "from src.config import settings; \
print(settings.mlflow_tracking_uri)"
```

Resultado:

```
http://44.215.228.113:5000
```

Esto confirma que el proyecto obtiene correctamente la dirección del Tracking Server desde la variable de entorno.

Adicionalmente, se realizó previamente una prueba de comunicación con el Tracking Server que permitió crear el experimento `preterm_infant_prediction` y registrar una ejecución de prueba.

## 15. Resultado

Se completó la configuración de un MLflow Tracking Server centralizado sobre AWS EC2. La solución utiliza una instancia `t3.small`, almacenamiento SQLite para los metadatos, almacenamiento local para artefactos, una Elastic IP para mantener un endpoint estable y `systemd` para iniciar automáticamente MLflow después del arranque de EC2.

El proyecto se conecta al servidor mediante la variable de entorno `MLFLOW_TRACKING_URI`, permitiendo que los integrantes del equipo utilicen el mismo Tracking Server sin modificar el código fuente.